Android App Bundleの動的機能モジュールに対するセキュリティ解析

佐藤 優樹[1,a)]　稲吉 弘樹[2]　掛井 将平[1]　齋藤 彰一[1]

概要：Android App Bundle では，Play Feature Delivery（PFD）により，アプリの一部機能を動的機能モジュールとして実行時に配信できる．一方，既存の Android アプリ解析手法は Base モジュールのみを解析対象としているため，後から配信されるモジュールを見落とす可能性がある．本稿では，Base モジュールのみを解析した場合と，動的機能モジュールも含めて解析した場合を比較し，Permission，Tracker，および静的テイント解析の結果に生じる差を評価する．APKPure から 764 アプリを収集し，15 アプリから 124 個の動的機能モジュールを取得した．さらに，VirusTotal で複数エンジンに検知されたアプリを対象に，動的機能モジュールの利用事例を調査した．評価の結果，Permission・Tracker は解析結果が変化しなかったが，静的テイント解析では Leak の検出数が増加した．以上から，Base モジュールのみを対象とする解析では，動的機能モジュールに含まれる情報フローを見落とす可能性があり，PFD を考慮した解析の必要性を示した．

キーワード：Play Feature Delivery, On-demand モジュール, Android App Bundle

Security Analysis of On-Demand Dynamic Feature Modules in Android App Bundles

Yuki Sato[1,a)]　Hiroki Inayoshi[2]　Shohei Kakei[1]　Shoichi Saito[1]

Abstract: Android App Bundles support runtime delivery of application features through Play Feature Delivery (PFD), which distributes application code as dynamic feature modules. Existing Android application analysis approaches primarily analyze only the base module and may therefore overlook modules delivered after installation. We compare the results of analyzing only the base module with those of analyzing both the base and dynamic feature modules, evaluating permission analysis, tracker detection, and static taint analysis. We collected 764 applications from APKPure and obtained 124 dynamic feature modules from 15 applications. We also investigated the use of dynamic feature modules in applications detected by multiple antivirus engines on VirusTotal. Permission analysis and tracker detection produced no differences, whereas static taint analysis detected more leaks. These results demonstrate that analyses targeting only the base module may miss information flows in dynamic feature modules, highlighting the importance of considering PFD.

Keywords: Play Feature Delivery, On-demand module, Android App Bundle

1. はじめに

従来，Android アプリは Android Package Kit（APK）形式で Google Play にアップロードされ，ユーザーへ配布されてきた．しかし，近年では，100 万以上のアプリが Android App Bundle（AAB）形式で Google Play にアップロードされている [1]．2018 年 5 月に Google が AAB を発表し，2021 年 8 月からは Google Play への新規アプリのアップロード形式として AAB が必須となった [2]．AAB

[1] 名古屋工業大学 Nagoya Institute of Technology
[2] 岡山大学 Okayama University
[a)] y.sato.250@nitech.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

でアップロードされたアプリは，Google Play がユーザーの端末環境に応じて APK を生成・配布する [3].

さらに，AAB でアップロードされたアプリは，Google Play を介してアプリの機能をモジュール単位で配信する Play Feature Delivery（PFD） [4] を利用できる．アプリを構成するモジュールは Base モジュールと機能モジュールに大別される．Base モジュールはアプリの起動に必要なコードおよび基本機能を提供し，初回インストール時に必ず配信される．一方，機能モジュールの一種である On-demand モジュールは，初回インストール時には配信されず，必要になったタイミングでダウンロード・インストールされる．

従来の Android アプリに対するセキュリティ解析では，初回インストール時に配信される単一の APK を解析対象とすることが一般的であった．しかし，On-demand モジュールを含むアプリでは，一部のコードが後から取得されるため，単一の APK のみを解析対象とする従来の解析では，コードを十分に解析できない可能性がある．

さらに，Android アプリ研究で広く利用されるデータセットである AndroZoo [5] も，2026 年 7 月 20 日時点では Base モジュールのみを収録対象としており，PFD の機能モジュールには対応していない．そのため，AndroZoo を用いた既存研究においても，On-demand モジュールが解析対象から欠落している可能性がある．

PFD や AAB に関する実態調査 [6] は報告されているものの，On-demand モジュールが既存の Android アプリのセキュリティ解析に与える影響については十分に調査されていない．そこで，本稿では，Base モジュールのみを解析した場合と，Base モジュールおよび On-demand モジュールを解析した場合を比較し，Permission 解析，Tracker 解析，および静的テイント解析を対象として，On-demand モジュールが既存の Android アプリのセキュリティ解析に与える影響を明らかにすることを目的とする．

本稿の構成は以下のとおりである．2 章で背景を述べ，3 章で関連研究を紹介する．4 章では評価に用いるデータセットの収集方法について述べ，5 章では比較対象と評価に用いた解析手法について述べる．6 章では評価結果と考察を示し，7 章で悪性アプリにおける On-demand モジュールの利用事例を紹介する．最後に，8 章で本稿のまとめについて述べる．

2. 背景

本章では，本稿に関連する技術的背景として，Google Play におけるアプリサイズの制限，APK と AAB の違い，PFD，および On-demand モジュールのインストール方法について述べる．

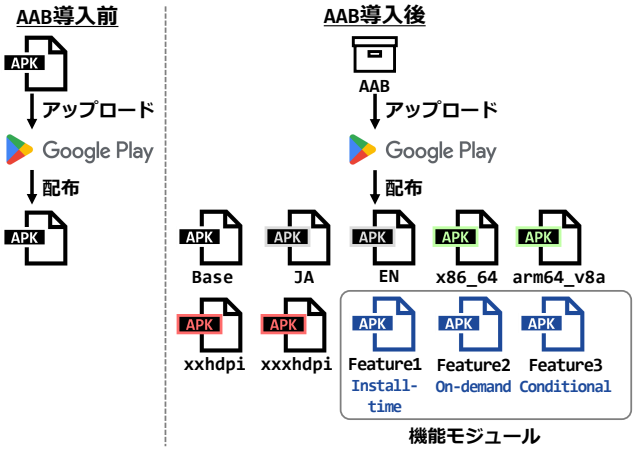


図 1 AAB 導入前後の Google Play のアプリ配信方式の違い
Fig. 1 Difference in app distribution methods on Google Play before and after the introduction of AAB

2.1 Google Play におけるアプリサイズの制限

Google Play では，アプリサイズが大きくなるとインストール成功率が低下し，アンインストール率が増える可能性があるため，可能な限りアプリサイズを小さく保つことが推奨されている [7]．AAB では，Google Play からダウンロードされる圧縮済みの各モジュールのサイズは 200 MB までとされており [3]，それを超えるアプリでは PFD や Play Asset Delivery [8] を利用することで，必要な機能やアセットを後から配信できる．

2.2 APK と AAB

図 1 に，AAB 導入前後の Google Play のアプリ配信方式の違いを示す．APK は，Android アプリをインストールおよび実行するためのパッケージ形式である．APK には，アプリの実行に必要なコード，リソース，`AndroidManifest.xml` などが含まれる．従来，開発者は単一の APK を Google Play へアップロードし，Google Play はその APK をユーザーへ配布していた．

一方，AAB は，アプリを構成するモジュールやリソースを含むアーカイブ形式である．AAB は，APK に代わるアプリのアップロード形式として 2018 年 5 月に Google によって導入され，2021 年 8 月からは Google Play への新規アプリのアップロード形式として必須となった [2]．端末の CPU アーキテクチャや画面密度，言語設定などの情報に応じて Google Play が APK を生成し，ユーザーへ配布する [3]．図 1 の `arm64_v8a` や `x86_64` は CPU アーキテクチャに応じたモジュール，`xxhdpi` や `xxxhdpi` は画面密度に応じたモジュール，`JA` や `EN` は言語設定に応じたモジュールを示す．このように，AAB の導入によりアプリサイズを削減できるようになった [9]．

© 2026 Information Processing Society of Japan

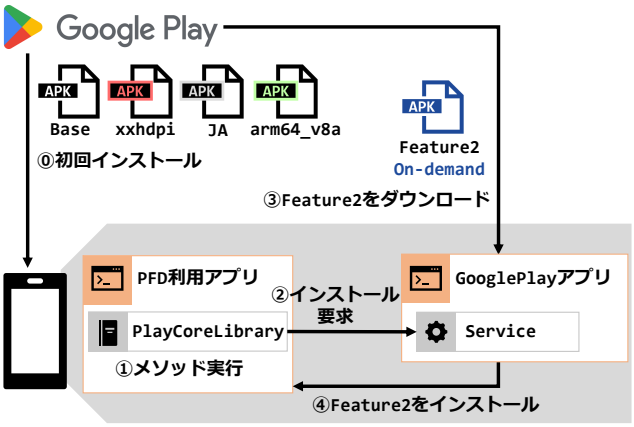


図 2 On-demand モジュールのインストール手順
Fig. 2 Installation procedure of On-demand modules

2.3 Play Feature Delivery の配信方式

PFD は，Google Play を介してアプリの機能をモジュール単位で配信する仕組みである [4]．PFD では，アプリは Base モジュールおよび複数の機能モジュールから構成される．Base モジュールは，アプリの起動に必要なコードおよび基本機能を提供するモジュールであり，初回インストール時に必ず配信される．機能モジュールは，配信方法に応じて以下の 3 種類に分類される．

- Install-time モジュール: Base モジュールと同時にインストールされるモジュールである．
- On-demand モジュール: アプリの実行中に必要となったタイミングでダウンロードおよびインストールされるモジュールである．
- Conditional モジュール: 国・地域，Android バージョン，ハードウェア機能など，指定した条件を満たす場合にのみ，Base モジュールと同時にインストールされるモジュールである．

これらの配信方式を組み合わせることで，必要な機能のみをユーザーへ提供し，初回インストール時のアプリサイズを削減できる．なお，PFD では，Base モジュールおよび各機能モジュールは，いずれも AAB に含まれた状態で Google Play へアップロードされる．このように，On-demand モジュールはアプリ公開後に開発者が任意のコードを追加する仕組みではない．モジュールの内容を変更するには AAB を更新し，公開し直す必要がある．

2.4 On-demand モジュールのインストール方法

アプリは，Google が提供する Play Core Library を利用して，On-demand モジュールのインストールを要求する．**図 2** に On-demand モジュールのインストール手順を示す．On-demand モジュールは，以下の流れで端末へインストールされる．

(1) アプリ内で Play Core Library のインストール要求メソッドが実行される
(2) Google Play アプリが提供するサービスに対し，Play Core Library がインストール要求を送信する
(3) Google Play アプリが指定された On-demand モジュールを Google Play からダウンロードする
(4) ダウンロードした On-demand モジュールを端末へインストールする

On-demand モジュールは，実際にアプリを操作することでインストールできる．しかし，どの操作でどのモジュールが要求されるかはアプリごとに異なる．そのため，多数のアプリを手動で操作し，On-demand モジュールを収集する場合は多くの時間と労力を要する．

3. 関連研究

本章では，AAB および PFD に関する研究，Android アプリの静的解析に関する研究，および Android アプリデータセットについて述べる．

3.1 AAB および PFD に関する研究

Tang ら [6] は Google Play で公開されている 20 万以上のアプリを対象に，AAB の採用状況や機能モジュールの利用実態を調査した．さらに，同研究では，機能モジュールを収集するための AppBundleExplorer を提案し，機能モジュールや動的アセットの設計手法について分析している．

また，Ghorbani ら [10] は，On-demand モジュールを対象としたテスト支援手法である DeltaDroid を提案した．DeltaDroid は，On-demand モジュールのインストール失敗を引き起こす状況を体系化し，その状況を再現するテストケースを自動生成することで，On-demand モジュールの品質保証を支援することを目的としている．

これらの研究では，AAB や PFD の利用実態の調査や On-demand モジュールの品質保証が主な対象であり，On-demand モジュールが既存の Android アプリのセキュリティ解析に与える影響については検討されていない．本稿では，モジュール欠落が既存のセキュリティ解析結果へ与える影響を評価する．

3.2 Android アプリの静的解析に関する研究

Android アプリに対する静的解析では，情報漏えいを検出する静的テイント解析手法である FlowDroid [11] をはじめ，多くの解析手法が提案されている．既存の Android アプリの静的解析手法の多くは，従来形式でアップロードされたアプリを解析対象とすることを前提としている．しかし，PFD を利用するアプリでは，On-demand モジュールが後からインストールされるため，Base モジュールのみを対象とした解析ではアプリ全体を十分に解析できない可能性がある．本稿では，Base モジュールのみを解析した場合と，Base モジュールおよび On-demand モジュールを解析

© 2026 Information Processing Society of Japan

した場合を比較することで，On-demand モジュールが既存の Android アプリのセキュリティ解析に与える影響を評価する．

Dynamic Code Loading を対象とした研究として，実行時に読み込まれたコードを取得し，静的テイント解析によって情報漏えいを検出する手法 [12] や，動的に読み込まれたコードに含まれるアプリ間通信を検出する手法 [13] が提案されている．これらの研究では，Google Play を介さず外部から取得し，DexClassLoader 等を用いて読み込まれる Java バイトコードを解析対象としている．一方，本稿で対象とする On-demand モジュールは，Google Play を介して配信される PFD の機能であり，Dynamic Code Loading とは配信・実行の仕組みが異なる．

3.3 Android アプリデータセット

AndroZoo [5] は Android アプリの研究で広く利用されるデータセットである．また，AndroZoo を基盤として構築されたデータセットも提案されている [14]．AndroZoo は現在も継続的に運用・拡張されている Android アプリデータセットであるが，Base モジュールのみを収録対象としており，PFD の機能モジュールには対応していない [*1]．そのため，AndroZoo を基盤として構築されたデータセットも，On-demand モジュールを含まない．

4. データセット

本稿では，データセットとして APKPure のアプリを利用した．APKPure はサードパーティアプリストアであり，APK をアップロードできるフォームを公開している [15]．また，APKPure は，アップロードされた APK について，公開前に署名の検証およびレビューを実施していると説明している [16]．APKPure を調査した結果，Base モジュールに加えて On-demand モジュールも公開されているアプリの存在を確認した．

APKPure では，同一アプリに対して複数のバージョンの APK が公開されている．PFD は初回インストールサイズの削減を目的として On-demand モジュールを導入していることから，本稿では On-demand モジュールが同梱されている可能性を高めるため，アプリサイズが大きいバージョンを優先してダウンロードした．また，公開されている仕様 [4] に従って，各機能モジュールに含まれる `AndroidManifest.xml` を解析し，配信方式を判定した．具体的には，<`dist:delivery`> 要素に <`dist:on-demand`> 要素が存在する場合に，当該モジュールを On-demand モジュールとして識別した．加えて，<`dist:module`> 要素の `dist:onDemand` 属性が `true` である場合も，On-demand モジュールとして識別した．

[*1] AndroZoo 管理者へ問い合わせたところ，2026 年 7 月 20 日時点では Base モジュールのみを収録対象としているとの回答を得た．

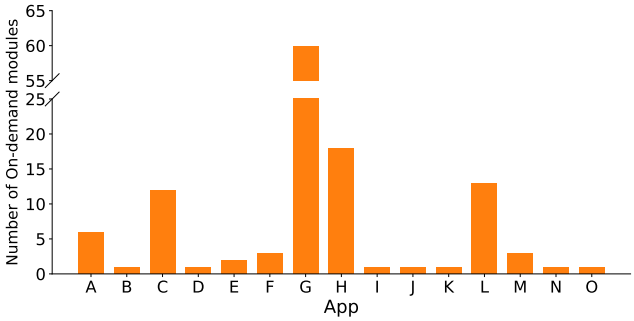


図 3 アプリごとの On-demand モジュール数
Fig. 3 Number of On-demand modules per app

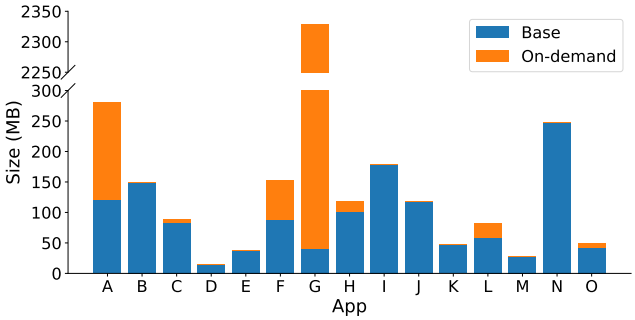


図 4 アプリごとの Base モジュールおよび On-demand モジュールのサイズ
Fig. 4 Size of Base module and On-demand modules per app

モジュールとして識別した．

その結果，APKPure から計 764 個のアプリを収集し，そのうち 15 アプリから合計 124 個の On-demand モジュールを取得した．**図 3** に各アプリが含む On-demand モジュール数を示す．15 アプリ中 7 アプリは On-demand モジュールを 1 個のみ含み，モジュール数の中央値は 2 個であった．一方，アプリ G は 60 個の On-demand モジュールを含み，全 124 モジュールの 48.4%を占めていた．このように，On-demand モジュール数は一部のアプリに大きく偏っている．次に，**図 4** に各アプリの Base モジュールおよび On-demand モジュールのサイズを示す．アプリサイズの最小値は 15.0 MB，最大値は 2328.5 MB，平均サイズは 261.5 MB，中央値は 117.8 MB であった．以降では，これらを評価対象として用いる．なお，図 3，図 4 および**表 1** で用いるアプリ識別子 A–O は，同一のアプリに対応する．

5. 評価方法

評価に用いた解析環境は以下の通りである．

- CPU: AMD Ryzen 9 7900X（4.7GHz，12 コア，24 スレッド）
- RAM: 128GB
- OS: Ubuntu Desktop Linux 24.04 LTS

図 5 に，本稿で行った評価の概要を示す．本章では，比較対象と評価に用いた Android アプリの解析手法について

© 2026 Information Processing Society of Japan

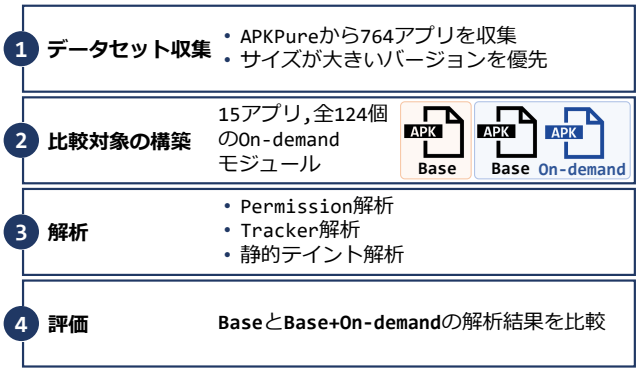


図 5 本稿で行った評価の概要
Fig. 5 Overview of the evaluation conducted in this paper

述べる．

5.1 比較対象

On-demand モジュールが既存の Android アプリ解析に与える影響を評価するため，本稿では Base モジュールのみを解析対象とした場合と，Base モジュールおよび On-demand モジュールを解析対象とした場合を比較する．両者に対して，Permission 解析，Tracker 解析，および静的テイント解析を行う．なお，FlowDroid は 1 つの APK を入力とするため，前処理として，APKEditor [17] を用いて Base モジュールと On-demand モジュールを統合する．

5.2 解析手法

評価に用いた Android アプリの解析手法について述べる．

5.2.1 Permission 解析

Android アプリにおける Permission は，アプリが位置情報やカメラ，ストレージなどの保護されたリソースへアクセスするために必要な権限であり，各 APK の `AndroidManifest.xml` にて宣言される．本稿では，各 APK が要求する Permission を抽出し，比較対象間で差分が生じるかを評価する．

5.2.2 Tracker 解析

Tracker は，広告配信や利用状況の分析，アプリのクラッシュ報告などを目的として情報を収集するライブラリの総称である．Exodus Privacy [18] は，Android アプリに含まれる Tracker を検出・識別するツールである．本稿では，Exodus Privacy を用いて検出された Tracker の種類および数を比較する．

5.2.3 静的テイント解析

Android アプリに対するテイント解析には，動的テイント解析と静的テイント解析が存在する．動的テイント解析では，実行時に観測されたデータフローを解析対象とするため，On-demand モジュールがインストールされ，かつ On-demand モジュール内のコードが実行されなければ，そのコードは解析対象とならない．一方，静的テイント解析では，コード全体を対象としてデータフローを解析できるため，On-demand モジュールを含めた解析に適している．そこで，本稿では Android 向け静的テイント解析ツールとして広く利用されている FlowDroid を用いて情報漏えい（Leak）の検出を行う．

FlowDroid は，あらかじめ定義された Source・Sink 間のデータフローを解析し，Leak の可能性を検出する．Google Play のデータセーフティ申告で対象の 14 データカテゴリについて，その取得に用いられる API を独自に収集し，Source として定義する．また，通信に関わる API を収集し，Sink として定義する．

Base モジュールから On-demand モジュールのメソッド呼び出しにはリフレクションが利用される．予備実験として複数のテストアプリを作成し，解析した結果，FlowDroid はリフレクションによる `static` メソッド呼び出しを追跡できないことを確認した．これは，FlowDroid の基盤技術である Soot がリフレクションを解析できないことに起因する [19]．したがって，Base モジュールからリフレクションを介して呼び出される On-demand モジュールの一部の情報フローは検出できていない可能性がある．本稿では，FlowDroid によって検出された Leak の数を比較する．

6. 評価結果と考察

6.1 Permission 解析

15 アプリすべてにおいて，Base モジュールのみを解析した場合と On-demand モジュールを含めて解析した場合で，検出される Permission に差は確認されなかった．On-demand モジュールで宣言されている Permission は，すべて Base モジュールでも宣言されていた．

そこで，この結果を検証するために追加実験を行った．具体的には，Base モジュールには Permission を宣言せず，On-demand モジュールにのみ Permission を宣言したテストアプリから AAB を生成し，Base モジュールの `AndroidManifest.xml` を調査した．その結果，On-demand モジュールにのみ宣言した Permission が，Base モジュールの `AndroidManifest.xml` に含まれていた．

以上から，Base モジュールのみを調査した場合でも，全モジュールが宣言する Permission を把握できると考えられる．したがって，Permission 解析に対するモジュール欠落の影響は限定的である．

ただし，Base モジュールの `AndroidManifest.xml` の調査だけでは，Permission を利用する処理がどのモジュールに存在するかを判別できない．そのため，Permission の正確な利用実態を明らかにするには各モジュールの解析が必要である．

6.2 Tracker 解析

15 アプリすべてにおいて，On-demand モジュールを解

© 2026 Information Processing Society of Japan

表 1　FlowDroid による静的テイント解析結果
Table 1　Results of static taint analysis using FlowDroid

	Source		Sink		解析状態		Leak	
アプリ	Base	Base+On-demand	Base	Base+On-demand	Base	Base+On-demand	Base	Base+On-demand
A	–	483	–	341	CG-OOM	完了	–	6
B	–	–	–	–	CG-OOM	CG-OOM	–	–
C	210	219	273	288	完了	完了	15	15
D	229	229	238	238	完了	完了	1	1
E	5094	7873	235	426	完了	完了	126	220
F	10	10	2	2	完了	完了	0	0
G	68	68	188	188	TA-TO	TA-TO	–	–
H	–	–	–	–	CG-TO	CG-TO	–	–
I	116	116	284	284	TA-TO	TA-TO	–	–
J	–	–	–	–	CG-TO	CG-TO	–	–
K	1765	1765	150	150	TA-TO	TA-TO	–	–
L	190	198	488	496	TA-TO	TA-TO	–	–
M	13	13	30	30	完了	完了	0	0
N	–	–	–	–	CG-OOM	CG-OOM	–	–
O	69	69	402	402	TA-TO	TA-TO	–	–

CG-OOM：CallGraph 生成時の OutOfMemoryError，CG-TO：CallGraph 生成時の Timeout，TA-TO：テイント解析時の Timeout.

析対象に追加することで新たに検出される Tracker は確認できなかった．On-demand モジュールから Tracker が検出されたアプリは存在したものの，これらの Tracker はいずれも Base モジュールからも検出された．

この解析の過程で，Install-time モジュールに，Base モジュールからは検出されない Tracker が含まれる事例を確認した．アプリ E の Install-time モジュールから，Base モジュールでは検出されなかった 2 つの Tracker が検出された．Exodus Privacy では，1 つは広告を目的とする Tracker として分類されており，もう 1 つは分析を目的とする Tracker として分類されている．この事例は，Install-time 配信や On-demand 配信といった配信方式に関わらず，Base モジュールのみを対象とする解析では，機能モジュールに含まれる Tracker を見落とし得ることを示している．

6.3　静的テイント解析

表 1 に，FlowDroid による静的テイント解析の結果を示す．表中の “–” は，当該指標の値を取得できなかったことを表す．15 アプリのうち，Base モジュールのみの場合と On-demand モジュールを含む場合の双方で解析が完了したアプリは 5 アプリであった．残る 10 アプリでは，少なくとも一方の解析において，タイムアウトまたはメモリ不足により解析を完了できなかった．

解析が完了した 5 アプリのうち，4 アプリでは Leak 数に変化がなかったが，アプリ E では，Base モジュールのみの場合に 126 件であった Leak が，On-demand モジュールを含めることで 220 件に増加した．また，アプリ C では，On-demand モジュールを含めることで Source 数が 210 から 219，Sink 数が 273 から 288 に増加したものの，Leak 数は 15 件で変化しなかった．

このように, Base モジュールのみの解析では, On-demand モジュールに含まれる Source・Sink 間の情報フローを見落とす場合があることが示された．一方，FlowDroid ではリフレクション呼び出しを追跡できないため，モジュール間フローの見落としが生じる可能性がある．加えて，大規模な実世界アプリでは計算時間とメモリ消費が制約となり，解析に失敗する場合がある．

7.　悪性アプリにおける On-demand モジュールの利用事例

本章では，On-demand モジュールに関する調査を通じて得られた，悪性アプリにおける On-demand モジュールの利用事例について述べる．

7.1　調査方法

図 6 に，本調査の概要を示す．以下の手順で調査を行う．

(1) AndroZoo から VirusTotal における検知数が 4 以上の Android アプリを収集し，Play Core Library を含む可能性が高いアプリをリストアップする．VirusTotal における検知数の閾値は，悪性アプリのラベル付けに 4 以上を用いる先行研究 [20] に従って設定する．

(2) 抽出したアプリの Base モジュールを jadx [21] でデコンパイルする．Play Core Library が提供する `addModule` メソッドおよび `deferredInstall` メソッドの呼び出し箇所を特定し，各メソッドに指定される On-demand

© 2026 Information Processing Society of Japan

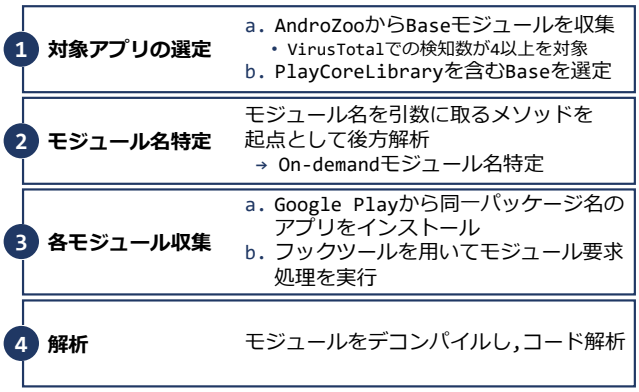


図 6 悪性アプリにおける On-demand モジュールの利用事例調査の概要

Fig. 6 Overview of the case study on the usage of On-demand modules in malicious apps

モジュール名を手動による後方解析によって特定する．

(3) 同一パッケージ名のアプリを Google Play からインストールする．

(4) フックツールを用いて，On-demand モジュールのインストール要求を行う処理を実行し，On-demand モジュールを取得する．

(5) 取得した On-demand モジュールに対して，jadx でデコンパイルし，実装されている機能を手動で分析する．

手順（3）において，Google Play からアプリをインストールする理由は，サイドローディングでインストールされた Base モジュールは，Google Play から On-demand モジュールを取得できないためである．サイドローディングしたアプリが，On-demand モジュールのインストールを要求すると，`APP_NOT_OWNED`（エラーコード −15）が返される．したがって，AndroZoo からダウンロードした Base モジュールのみを用いて On-demand モジュールを取得することはできず，Google Play からアプリをインストールする必要がある．

7.2 利用事例

本事例調査により，調査対象となりうるアプリを 1 件特定した．当該アプリは，2026 年 7 月 31 日時点で，4 ベンダーに検知されていた．

調査の結果，動画ダウンロード機能を実装した On-demand モジュールを確認した．当該モジュールには，Android 上で音声および動画をダウンロードするためのオープンソースソフトウェアが含まれていた．また，当該アプリの公式 Web サイトでは，「主要な動画共有サイトから動画をダウンロードし，オフライン再生できる」機能を提供予定であることが説明されており，本解析結果と整合している．本ライブラリを用いたダウンロードは，YouTube が提供する正規のオフライン再生機能を利用するものではなく，利用方法によっては YouTube の利用規約に抵触する可能性がある．

Base モジュールのサイズは 15 MB 以下，On-demand モジュールのサイズは 5 MB 以下であり，Google Play における Base モジュールのサイズ上限である 200 MB を大きく下回っている．当該アプリの Base モジュールは小規模であり，アプリサイズ削減という PFD 本来の目的を考慮すると，動画ダウンロード機能を遅延インストールすることによるサイズ削減効果は限定的であると考えられる．この事例から，Base モジュールのみの解析では，主要機能の実装を見落とす可能性があると言える．

また，本調査では，On-demand モジュールを利用する悪性アプリの収集は容易ではなかった．悪性アプリが Google Play から削除されている場合が多く，AndroZoo に収録されている悪性アプリであっても，同一パッケージ名のアプリを Google Play から取得できない事例が多数確認された．そのため，On-demand モジュールを含む悪性アプリの大規模な収集には一定の制約があることが明らかとなった．

8. まとめ

本稿では，APKPure から 764 アプリを収集し，そのうち 15 アプリから得た 124 個の On-demand モジュールを対象として，Base モジュールのみの場合と On-demand モジュールを含む場合のセキュリティ解析結果を比較した．Permission 解析および Tracker 解析では差が確認されなかった一方，静的テイント解析では，アプリ E の Leak が 126 件から 220 件へ増加した．また，利用事例から，Base モジュールのみの解析では主要な機能の実装を見落とす可能性があることを示した．

以上より，Base モジュールのみを対象とする解析では，On-demand モジュールに含まれる情報フローや機能を十分に把握できない場合がある．今後は，機能モジュールを Google Play から継続的に収集する基盤の整備と，複数モジュールから構成される大規模アプリにも適用可能な解析手法が必要である．

参考文献

[1] Google: Android App Bundle frequently asked questions, (online), available from ⟨`https://developer.android.com/guide/app-bundle/faq`⟩ (accessed 2026-07-18).

[2] Google: Inspect app versions on the Latest releases and bundles page - Play Console Help, (online), available from ⟨`https://support.google.com/googleplay/android-developer/answer/9844279`⟩ (accessed 2026-07-18).

[3] Google: About Android App Bundles, (online), available from ⟨`https://developer.android.com/guide/app-bundle`⟩ (accessed 2026-07-18).

[4] Google: Overview of Play Feature Delivery, (online), available from ⟨`https://developer.android.com/guide/playcore/feature-delivery`⟩ (accessed 2026-07-18).

[5] Allix, K., Bissyandé, T. F., Klein, J. and Le Traon,

© 2026 Information Processing Society of Japan

Y.: AndroZoo: Collecting Millions of Android Apps for the Research Community, *Proceedings of the 13th International Conference on Mining Software Repositories*, MSR '16, New York, NY, USA, ACM, pp. 468–471 (online), DOI: 10.1145/2901739.2903508 (2016).

[6] Tang, Y., Luo, X. and Zhou, Y.: A Systematic Study on Real-World Android App Bundles, *IEEE Transactions on Software Engineering*, Vol. 51, No. 5, pp. 1615–1628 (online), DOI: 10.1109/TSE.2025.3560026 (2025).

[7] Google: Reduce your app size, (online), available from ⟨`https://developer.android.com/topic/performance/reduce-apk-size`⟩ (accessed 2026-07-20).

[8] Google: Play Asset Delivery, (online), available from ⟨`https://developer.android.com/guide/playcore/asset-delivery`⟩ (accessed 2026-07-20).

[9] Google: I/O 2018: Everything new in the Google Play Console, (online), available from ⟨`https://android-developers.googleblog.com/2018/05/io-2018-everything-new-in-google-play.html`⟩ (accessed 2026-07-18).

[10] Ghorbani, N., Jabbarvand, R., Salehnamadi, N., Garcia, J. and Malek, S.: DeltaDroid: Dynamic Delivery Testing in Android, *ACM Trans. Softw. Eng. Methodol.*, Vol. 32, No. 4 (online), DOI: 10.1145/3563213 (2023).

[11] Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D. and McDaniel, P.: FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps, *ACM sigplan notices*, Vol. 49, No. 6, pp. 259–269 (2014).

[12] Yang, Y., Luo, W., Pei, Y., Pan, M. and Zhang, T.: Execution Enhanced Static Detection of Android Privacy Leakage Hidden by Dynamic Class Loading, *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 1, pp. 149–158 (online), DOI: 10.1109/COMPSAC.2019.00029 (2019).

[13] Alhanahnah, M., Yan, Q., Bagheri, H., Zhou, H., Tsutano, Y., Srisa-An, W. and Luo, X.: DINA: Detecting Hidden Android Inter-App Communication in Dynamic Loaded Code, *Trans. Info. For. Sec.*, Vol. 15, p. 2782–2797 (online), DOI: 10.1109/TIFS.2020.2976556 (2020).

[14] Bragança, H., Kreutz, D., Rocha, V., Assolin, J. and Feitosa, E.: MH-1M: A 1.34 Million-Sample Multi-Feature Android Malware Dataset with Rich Metadata, *Scientific Data*, Vol. 13, No. 1, p. 153 (online), DOI: 10.1038/s41597-025-06469-5 (2025).

[15] APKPure: Submit your Android Apps and games - Upload .APK File (Android App) at APKPure, (online), available from ⟨`https://apkpure.com/submit-apk`⟩ (accessed 2026-07-17).

[16] APKPure: Why we can guarantee APK 100% safe?, (online), available from ⟨`https://apkpure.net/faq-safe.html`⟩ (accessed 2026-07-17).

[17] REAndroid: APKEditor: Powerful android apk editor - aapt/aapt2 independent, (online), available from ⟨`https://github.com/REAndroid/APKEditor`⟩ (accessed 2026-07-17).

[18] ExodusPrivacy: Exodus Privacy, (online), available from ⟨`https://exodus-privacy.eu.org/en/`⟩ (accessed 2026-07-17).

[19] secure-software engineering: [FN] Tracking fails for static method reflection · Issue #875 · secure-software-engineering/FlowDroid, (online), available from ⟨`https://github.com/secure-software-engineering/FlowDroid/issues/875`⟩ (accessed 2026-07-20).

[20] Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J. and Cavallaro, L.: TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time, *28th USENIX Security Symposium (USENIX Security 19)*, Santa Clara, CA, USENIX Association, pp. 729–746 (online), available from ⟨`https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury`⟩ (2019).

[21] skylot: jadx: Dex to Java decompiler, (online), available from ⟨`https://github.com/skylot/jadx`⟩ (accessed 2026-07-21).

© 2026 Information Processing Society of Japan